

CGROUPS and CPU/Memory Resource Utilization Hands On

Linux namespaces isolate different aspects of a process's environment:

- **PID** → process IDs, what containers use so they don't see processes from the host.
- **NET** → networking (interfaces, IPs) allow each container to have its own network stack.
- **IPC** → inter-process communication (shared memory, message queues), prevent processes in one container from interfering with shared memory in another.
- **MNT** → mount points and filesystem views, give processes their own view of the filesystem.
- **UTS** → hostname and domain name, Each container can have its own hostname.
- **USER** → user IDs and capabilities

- Create a container without any limits mentioned : `docker run -it --name no-limits ubuntu bash`

```
(base) labuser@ip-172-31-36-202:~$ docker run -it --name no-limits ubuntu bash
root@838fa9693080:/#
```

- Install stress package : `sudo apt update && apt-get install -y stress`

```
The following NEW packages will be installed:
  stress
0 upgraded, 1 newly installed, 0 to remove and 1 not upgraded.
Need to get 17.5 kB of archives.
After this operation, 92.2 kB of additional disk space will be used.
Get:1 http://ports.ubuntu.com/ubuntu-ports noble/universe arm64 stress arm64 1.0.7-1 [17.5 kB]
Fetched 17.5 kB in 0s (39.5 kB/s)
debconf: delaying package configuration, since apt-utils is not installed
Selecting previously unselected package stress.
(Reading database ... 4376 files and directories currently installed.)
Preparing to unpack .../stress_1.0.7-1_arm64.deb ...
Unpacking stress (1.0.7-1) ...
Setting up stress (1.0.7-1) ...
root@838fa9693080:/# _
```

- Start stress testing with 10 CPU instances for 30 sec : `stress --cpu 10 --timeout 30`

Monitor resource usage using top command :

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
12301	logstash	39	19	4144744	235016	22092	S	168.4	1.5	0:09.35	java
12273	root	20	0	2924	512	512	R	24.9	0.0	0:04.61	stress
12276	root	20	0	2924	512	512	R	24.3	0.0	0:04.14	stress
12275	root	20	0	2924	512	512	R	23.9	0.0	0:03.96	stress
12274	root	20	0	2924	512	512	R	22.3	0.0	0:04.44	stress
12277	root	20	0	2924	512	512	R	22.3	0.0	0:04.35	stress
12271	root	20	0	2924	512	512	R	21.3	0.0	0:04.27	stress
12272	root	20	0	2924	512	512	R	20.9	0.0	0:04.07	stress
12278	root	20	0	2924	512	512	R	20.9	0.0	0:04.14	stress
12279	root	20	0	2924	512	512	R	20.9	0.0	0:04.28	stress
12280	root	20	0	2924	512	512	R	20.6	0.0	0:04.13	stress
617	kibana	20	0	9843916	518776	48896	S	2.3	3.2	1:10.58	node

Check the processes : `ps -ef | grep stress` (1 parent process and 10 child processes)

```
(base) labuser@ip-172-31-36-202:~$ ps -ef | grep stress
root      12830   11441    0 06:25 pts/0    00:00:00 stress --cpu 10 --timeout 30
root      12831   12830   27 06:25 pts/0    00:00:02 stress --cpu 10 --timeout 30
root      12832   12830   25 06:25 pts/0    00:00:02 stress --cpu 10 --timeout 30
root      12833   12830   26 06:25 pts/0    00:00:02 stress --cpu 10 --timeout 30
root      12834   12830   28 06:25 pts/0    00:00:02 stress --cpu 10 --timeout 30
root      12835   12830   27 06:25 pts/0    00:00:02 stress --cpu 10 --timeout 30
root      12836   12830   26 06:25 pts/0    00:00:02 stress --cpu 10 --timeout 30
root      12837   12830   25 06:25 pts/0    00:00:02 stress --cpu 10 --timeout 30
root      12838   12830   27 06:25 pts/0    00:00:02 stress --cpu 10 --timeout 30
root      12839   12830   27 06:25 pts/0    00:00:02 stress --cpu 10 --timeout 30
root      12840   12830   24 06:25 pts/0    00:00:02 stress --cpu 10 --timeout 30
labuser   12913    8784    0 06:26 pts/1    00:00:00 grep --color=auto stress
```

- Now start memory stress test : `stress --vm 1 --vm-bytes 5000M --timeout 50`

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
13285	logstash	39	19	4144744	232372	22068	S	202.0	1.4	0:06.85	java
13195	root	20	0	5122928	1.7g	512	R	100.0	10.8	0:17.15	stress

- Launch another container with predefined limits : `docker run -it --name with-limits --memory=256m --cpus=0.5 ubuntu bash`

```
(base) labuser@ip-172-31-36-202:~$ docker run -it --name with-limits --memory=256m --cpus=0.5 ubuntu bash
root@833f8cefd60c:/# _
```

- Install stress package : `sudo apt update apt-get install -y stress`

```
The following NEW packages will be installed:
stress
0 upgraded, 1 newly installed, 0 to remove and 1 not upgraded.
Need to get 17.5 kB of archives.
After this operation, 92.2 kB of additional disk space will be used.
Get:1 http://ports.ubuntu.com/ubuntu-ports noble/universe arm64 stress arm64 1.0.7-1 [17.5 kB]
Fetched 17.5 kB in 1s (22.0 kB/s)
debconf: delaying package configuration, since apt-utils is not installed
Selecting previously unselected package stress.
(Reading database ... 4376 files and directories currently installed.)
Preparing to unpack .../stress_1.0.7-1_arm64.deb ...
Unpacking stress (1.0.7-1) ...
Setting up stress (1.0.7-1) ...
root@833f8cefd60c:/# _
```

- Start stress testing with 4 CPU instances for 10 sec (CPU more than defined limit but still container executes).

```
root@833f8cefd60c:/# stress --cpu 4 --timeout 10
stress: info: [159] dispatching hogs: 4 cpu, 0 io, 0 vm, 0 hdd
stress: info: [159] successful run completed in 10s
root@833f8cefd60c:/# _
```

top output on node :

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
15710	logstash	39	19	4212320	379228	22092	S	287.0	2.3	0:27.95	java
15773	root	20	0	2924	512	512	R	13.3	0.0	0:00.50	stress
15771	root	20	0	2924	512	512	R	12.3	0.0	0:00.47	stress
15774	root	20	0	2924	512	512	R	12.3	0.0	0:00.48	stress
15772	root	20	0	2924	512	512	R	11.6	0.0	0:00.48	stress

- Now start memory stress test with 500Mb memory size (more than defined 256Mb) :
`stress --vm 1 --vm-bytes 500M --timeout 10 # Will result in OutOfMemory Issue ( our limit was 256 mb).`

```
root@833f8cefd60c:/# stress --vm 1 --vm-bytes 500M --timeout 10
stress: info: [164] dispatching hogs: 0 cpu, 0 io, 1 vm, 0 hdd
stress: FAIL: [164] (425) <-- worker 165 got signal 9
stress: WARN: [164] (427) now reaping child worker processes
stress: FAIL: [164] (461) failed run completed in 0s
root@833f8cefd60c:/#
```

So if current memory usage exceeds predefined memory limit, container gets terminated by the controlled, while in case of CPU utilization exceeding its predefined limit, container continues to work (but with degraded/throttled state).

Example below explains logs inside a worker node when few containers had memory usage exceeding their limits and thus terminated because of OOM (OutOfMemory) error with Error Code 137.

```
[ 160.000000] Memory cgroup out of memory: Killed process 3202358 (multus-daemon) total-virt=1273824kB, anon-rss=15388kB, file-rss=29392kB, shmem-rss=0kB, UID=0 pgtables=216kB oom_score_adj=999
[ssm-user@ip-10-225-105-223 ~]$ dmesg -T | grep -i "killed process"
[Fri Mar 21 22:55:26 2025] Memory cgroup out of memory: Killed process 55390 (gnb_cu_cp_ue_co) total-virt=3206000kB, anon-rss=202421kB, file-rss=2358kB, shmem-rss=0kB, UID=0 pgtables=464kB oom_score_adj=997
[Fri Mar 20 02:47:37 2025] Memory cgroup out of memory: Killed process 16165 (multus-daemon) total-virt=127356kB, anon-rss=18228kB, file-rss=31184kB, shmem-rss=0kB, UID=0 pgtables=222kB oom_score_adj=999
[Fri May 30 06:45:24 2025] Memory cgroup out of memory: Killed process 1363891 (multus-daemon) total-virt=1273056kB, anon-rss=19124kB, file-rss=30928kB, shmem-rss=0kB, UID=0 pgtables=224kB oom_score_adj=999
[Tue Aug 5 10:13:49 2025] Memory cgroup out of memory: Killed process 14696 (defender) total-virt=1679132kB, anon-rss=284436kB, file-rss=86932kB, shmem-rss=0kB, UID=0 pgtables=1012kB oom_score_adj=999
[Fri Aug 15 02:29:06 2025] Memory cgroup out of memory: Killed process 3367304 (multus-daemon) total-virt=1273056kB, anon-rss=18904kB, file-rss=31120kB, shmem-rss=0kB, UID=0 pgtables=236kB oom_score_adj=999
[Fri Aug 15 02:29:25 2025] Memory cgroup out of memory: Killed process 3136189 (multus-daemon) total-virt=1271520kB, anon-rss=14000kB, file-rss=28944kB, shmem-rss=0kB, UID=0 pgtables=208kB oom_score_adj=999
[Fri Aug 15 02:29:45 2025] Memory cgroup out of memory: Killed process 3141925 (multus-daemon) total-virt=1273112kB, anon-rss=13908kB, file-rss=29200kB, shmem-rss=0kB, UID=0 pgtables=224kB oom_score_adj=999
[Fri Aug 15 02:29:47 2025] Memory cgroup out of memory: Killed process 3142460 (aws-cni) total-virt=1242508kB, anon-rss=7056kB, file-rss=11108kB, shmem-rss=0kB, UID=0 pgtables=144kB oom_score_adj=999
[Fri Aug 15 02:30:21 2025] Memory cgroup out of memory: Killed process 3145575 (multus-daemon) total-virt=1273824kB, anon-rss=16024kB, file-rss=29008kB, shmem-rss=0kB, UID=0 pgtables=228kB oom_score_adj=999
[Fri Aug 15 02:31:13 2025] Memory cgroup out of memory: Killed process 3150462 (multus-daemon) total-virt=1272288kB, anon-rss=16096kB, file-rss=29136kB, shmem-rss=0kB, UID=0 pgtables=216kB oom_score_adj=999
[Fri Aug 15 02:32:47 2025] Memory cgroup out of memory: Killed process 3159591 (multus-daemon) total-virt=1273056kB, anon-rss=19124kB, file-rss=29584kB, shmem-rss=0kB, UID=0 pgtables=228kB oom_score_adj=999
[Fri Aug 15 02:32:50 2025] Memory cgroup out of memory: Killed process 3159969 (aws-cni) total-virt=1243092kB, anon-rss=6724kB, file-rss=10364kB, shmem-rss=0kB, UID=0 pgtables=140kB oom_score_adj=999
[Fri Aug 15 02:35:43 2025] Memory cgroup out of memory: Killed process 3175188 (multus-daemon) total-virt=1271776kB, anon-rss=14268kB, file-rss=28204kB, shmem-rss=0kB, UID=0 pgtables=208kB oom_score_adj=999
[Fri Aug 15 02:40:56 2025] Memory cgroup out of memory: Killed process 3201358 (multus-daemon) total-virt=1271264kB, anon-rss=15388kB, file-rss=29392kB, shmem-rss=0kB, UID=0 pgtables=216kB oom_score_adj=999
[ssm-user@ip-10-225-105-223 ~]$ date
Fri Aug 15 04:09:16 UTC 2025
```

Docker Stack Deployment for 3 tier microservice based application

- To initiate the Docker Swarm : `docker swarm init`

```
(base) labuser@ip-172-31-23-58:~$ docker swarm init
Swarm initialized: current node (pymzmngz1eshe4hyl65t56vta) is now a manager.

To add a worker to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-4hhuv5d8bmpsfbchqhcjev39tc20xxrlwbzu4bj43psacy2wi4-ci1z1momrc3xbox12t18u0ldr 172.31.23.58:2377

To add a manager to this swarm, run 'docker swarm join-token manager' and follow the instructions.

(base) labuser@ip-172-31-23-58:~$
```

- Create a docker-compose file and deploy using stack : `docker stack deploy -c docker-compose.yml my-app-stack`

```
(base) labuser@ip-172-31-23-58:~$ docker stack deploy -c docker-compose.yml my-app-stack
Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Creating network my-app-stack_backend-network
Creating network my-app-stack_frontend-network
Creating network my-app-stack_db-network
Creating service my-app-stack_db
Creating service my-app-stack_backend
Creating service my-app-stack_frontend
(base) labuser@ip-172-31-23-58:~$
```

- Check resources created : `docker volume ls` `docker network ls`

```
(base) labuser@ip-172-31-23-58:~$ docker volume ls
DRIVER      VOLUME NAME
local       my-app-stack_db_data
(base) labuser@ip-172-31-23-58:~$ docker network ls
NETWORK ID    NAME                                DRIVER      SCOPE
aa31061754c7  bridge                            bridge      local
58390e76c828  docker_gwbridge                   bridge      local
ec734391f431  host                              host        local
2rc7gnpf21ni  ingress                           overlay     swarm
yh5f02s593pd  my-app-stack_backend-network      overlay     swarm
jen2v19qtv05  my-app-stack_db-network           overlay     swarm
om03pdn9769t  my-app-stack_frontend-network     overlay     swarm
4b98f92ed8ca  none                              null        local
```

- Check services(tasks) created by stack : `docker stack ps my-app-stack`

```
(base) labuser@ip-172-31-23-58:~$ docker stack services my-app-stack
ID            NAME                                MODE          REPLICAS  IMAGE              PORTS
w2blk135u9kr  my-app-stack_backend               replicated    3/10      openjdk:17-slim
xgt51dl63bfc  my-app-stack_db                    replicated    1/1       mysql:8.0
gf2n2ym5k3ox  my-app-stack_frontend               replicated    10/10     nginx:latest      *:8080->80/tcp
```

- Scale the service using scale command : `docker service scale my-app-stack_backend=2`

```
(base) labuser@ip-172-31-23-58:~$ docker service scale my-app-stack_backend=2
my-app-stack_backend scaled to 2
overall progress: 2 out of 2 tasks
1/2:
2/2:
verify: Service my-app-stack_backend converged
(base) labuser@ip-172-31-23-58:~$
```

- Cleanup the stack : `docker stack rm my-app-stack`

```
(base) labuser@ip-172-31-23-58:~$ docker stack rm my-app-stack
Removing service my-app-stack_backend
Removing service my-app-stack_db
Removing service my-app-stack_frontend
Removing network my-app-stack_db-network
Removing network my-app-stack_frontend-network
Removing network my-app-stack_backend-network
(base) labuser@ip-172-31-23-58:~$
```


Docker Healthcheck Hands On

- Create a docker-compose.yml file with below content and deploy the services :

```
version: "3.8"
services:
  web:
    image: nginx:alpine
    ports:
      - "8081:80"
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost/"]
      interval: 30s      # run healthcheck every 30 seconds
      timeout: 10s       # consider unhealthy if command takes >10s
      retries: 3          # mark as unhealthy after 3 consecutive failures
      start_period: 5s    # wait 5s before starting healthcheck
```

- Deploy the compose file in detached mode : docker compose up -d

```
(base) labuser@ip-172-31-23-58:~$ docker compose up -d
WARN[0000] /home/labuser/docker-compose.yml: the attribute `version` is obsolete, i
[+] Running 1/1
✓ Container labuser-web-1 Started
```

- Monitor the status of the container and it will show healthy after some time.

```
(base) labuser@ip-172-31-23-58:~$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS              PORTS                               NAMES
1b889602470c   nginx:alpine   "/docker-entrypoint..." About a minute ago Up 3 seconds (health: starting) 0.0.0.0:8080->80/tcp, :::8080->80/tcp labuser-web-1
(base) labuser@ip-172-31-23-58:~$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS              PORTS                               NAMES
1b889602470c   nginx:alpine   "/docker-entrypoint..." 2 minutes ago Up 26 seconds (healthy) 0.0.0.0:8080->80/tcp, :::8080->80/tcp labuser-web-1
(base) labuser@ip-172-31-23-58:~$
```